Jarvis Voice Assistant — Build Specification

> **Purpose of this document:** A complete specification for Claude Code to build a voice-activated AI assistant. This document describes *what* to build, *how* it should behave, and *what design decisions matter*. Implementation choices (libraries, code structure, error handling patterns) are left to Claude Code's judgment based on the user's environment.

---

1. Project Overview

Build a voice-activated AI assistant called **Jarvis** that runs on the user's laptop and is remotely controllable from a mobile companion app. The assistant should:

- Activate on the wake word "Jarvis"
- Transcribe spoken commands to text
- Use an LLM (Claude API) to understand intent and orchestrate tool calls
- Execute actions on the user's computer (system control, web tasks, productivity apps)
- Speak responses naturally
- Maintain conversational memory across sessions
- Present a futuristic, restrained UI showing real-time state
- Expose a local API so a mobile companion app can send commands

Target platforms: Laptop (priority — Windows/macOS/Linux), Android mobile companion (Phase 2 of build).

End user: Prathamesh, a CS student building this as a portfolio project. He prefers clean, minimal, professional aesthetics over flashy ones.

---

2. Architecture Principles

Follow these principles throughout:

- **Modular by capability.** Wake-word, audio I/O, LLM orchestration, and tool implementations should each be independent modules that can be swapped without rewriting the rest.
- **Tool-based extensibility.** New capabilities are added by writing a new tool function and registering it — not by editing the core orchestration loop.

- **Voice and text are equivalent inputs.** Any command that works by voice must also work via the text input field on the UI and via the mobile API. Design the brain layer to take text in and return text out; voice is just an adapter on top.
- **Separation of concerns:** the orchestrator decides *what* to do (LLM tool calling), the tools handle *how* to do it. Tools should be pure functions that take typed inputs and return strings.
- **Fail gracefully.** If a tool errors, the assistant should report the failure conversationally and continue, not crash.
- **Privacy-first defaults.** Prefer local processing (local STT) over cloud APIs unless cloud offers a clear quality benefit. Never log audio.

---

3. Tech Stack Decisions

These choices are recommended; Claude Code may substitute equivalents if a clear reason exists (e.g., platform incompatibility).

Layer	Recommended	Rationale
Language (backend)	Python 3.11+	Fits user's AI/ML background; best library ecosystem for voice
Wake word	Picovoice Porcupine	Free tier, runs offline, includes "jarvis" built-in
Speech-to-text	faster-whisper (local)	Free, private, accurate. "base" model is good speed/quality balance
LLM	Anthropic Claude API with tool use	Best-in-class tool orchestration
Text-to-speech	edge-tts	Free, natural-sounding voices, no API key needed
Audio playback	pygame.mixer or sounddevice	Reliable cross-platform
Memory store	ChromaDB with sentence-transformers	Local vector DB, no external service
Server	FastAPI + Uvicorn	Async, WebSocket support, clean API
UI (desktop)	Single-file HTML/CSS/JS served by FastAPI	No build step, matches user's preference for vanilla web
Mobile	React Native via Expo	Cross-platform, fast iteration
Browser automation	Playwright	More reliable than Selenium

Do not introduce a heavy frontend framework (Next.js, etc.) for the desktop UI — the user prefers vanilla HTML/CSS/JS for portfolio-style projects.

---

4. Build Phases

Build in this order. **Stop at the end of each phase** and let the user test before continuing. Each phase must produce something testable in isolation.

Phase 1: Voice Loop Foundation

The minimum viable loop: wake word triggers recording, recording is transcribed, transcription is echoed back via TTS.

Acceptance: User says "Jarvis", hears a confirmation tone or word, speaks a sentence, and hears Jarvis repeat it back.

Phase 2: LLM Brain with Tool Use

Replace the echo with Claude API. Claude should be able to call registered tool functions and respond conversationally. Start with one trivial tool (e.g., get current time) to verify the loop.

Acceptance: User asks "what time is it" — Claude calls the time tool and speaks the result naturally.

Phase 3: System Control Tools

Implement tools for: opening applications, closing applications, setting system volume, listing files in a directory, creating text notes in `~/Documents/JarvisNotes`.

Acceptance: Voice commands like "open Chrome", "set volume to 30 percent", "save a note about the meeting tomorrow" all work.

Phase 4: Web Tools

Implement: web search (via DuckDuckGo HTML or similar — no API key required), opening URLs in default browser, YouTube search shortcut, fetching readable text from a URL.

Acceptance: "Search the web for the latest AI news" returns a summary spoken aloud. "Open GitHub" launches the browser to github.com.

Phase 5: Productivity Tools (Gmail + Calendar)

Implement Google OAuth flow and tools for: listing recent unread emails (sender + subject), sending email, listing upcoming calendar events, creating calendar events.

Critical: Before sending an email or creating an event, Jarvis must verbally confirm the details with the user and wait for explicit confirmation.

Acceptance: "Read my unread emails" lists them. "Schedule a meeting with John tomorrow at 3pm" creates the event after confirmation.

Phase 6: Conversational Memory

Two layers:

1. **Session history** — already handled by the brain's conversation list.
2. **Long-term memory** — vector store of facts. Provide tools `remember_fact(fact)` and `recall_memory(query)`. Update the system prompt so Claude proactively recalls relevant memories at the start of conversations and saves things the user explicitly tells it to remember.

Acceptance: "Remember that I prefer my coffee black" stores the fact. In a new session, "how do I take my coffee" recalls it.

Phase 7: Desktop UI

A single-file HTML page served by the backend. See Section 6 for full design spec. Real-time state via WebSocket.

Acceptance: Browser opens to `http://localhost:8765`, shows the orb interface, displays live transcripts as the user speaks, orb animates differently for idle/listening/thinking/speaking states.

Phase 8: API Server + WebSocket Integration

Wrap everything in FastAPI. Expose:

- `POST /command` — text command, returns response (for mobile and testing)
- `WS /ws` — real-time bidirectional state updates and transcripts
- `GET /` — serves the desktop UI
- `POST /transcribe` — accepts an audio file, returns transcribed text (for mobile voice input)

The voice loop should run as a background async task in the same FastAPI process so UI events stay synchronized.

Acceptance: Curl to `/command` works. WebSocket events arrive in the UI when voice activates. Mobile app (next phase) can hit the API.

Phase 9: Mobile Companion App

React Native via Expo. Acts as a *remote control* — sends voice/text commands to the laptop server, displays responses. Does not run its own LLM or tools.

Acceptance: Phone on same Wi-Fi as laptop, app connects to laptop's local IP, user can type or speak a command and see/hear the response.

Phase 10: Polish

- Latency optimization (streaming Claude responses, starting TTS on first sentence)
- False activation handling (sensitivity tuning, optional cancel window)
- Authentication on the API (token-based, configurable via env var)
- README with setup instructions

---

5. Behavior Requirements

Voice interaction style

- Jarvis responses must be **concise and conversational** since they're spoken aloud. No bullet points, no headers, no markdown — those don't translate to speech. Aim for 1–3 sentences per response unless the user explicitly asks for detail.
- After completing actions, briefly confirm what was done. Don't over-explain.
- Match a calm, professional tone — not chirpy, not robotic.

Wake-and-listen flow

- After wake word fires: play a subtle confirmation cue (short tone or single word like "Yes?"), then start recording.
- Stop recording on silence detection (about 1.5 seconds of silence) or after a max duration (~10 seconds).
- If transcription is empty or just noise, return to idle without sending to the LLM.

Confirmation rules (important)

Jarvis must **verbally confirm** before executing:

- Sending email
- Creating calendar events
- Closing applications by name (could close wrong app)
- Any destructive file operation

For idempotent or trivially reversible actions (opening apps, web searches, volume changes), no confirmation needed.

System prompt for the LLM

The brain's system prompt should establish:

- Identity ("You are Jarvis, a voice assistant for Prathamesh")
- Voice constraint ("Responses are spoken aloud — keep them concise, no markdown")
- Tool usage ("Use available tools to perform actions; don't just describe what you would do")
- Memory awareness ("At the start of conversations referencing past context, use recall_memory. When the user shares preferences or facts to remember, use remember_fact")

- Confirmation rules (list the destructive actions above)

Error handling

- Tool errors: catch, return a string error message to the LLM, let the LLM phrase it for the user. Never crash the loop.
- LLM API errors: catch, speak a friendly fallback ("Sorry, I had trouble thinking just now"), return to idle.
- Wake word detector should auto-restart if it crashes.

---

6. UI Design Specification

The user has strong design preferences. Follow these exactly.

Aesthetic direction

"Apple meets NASA mission control." Restrained, professional, futuristic but not flashy. Reference: clean dark interfaces with a single accent, generous whitespace, monospace typography for technical/system content.

Color palette (strict — do not introduce additional colors)

- Background: `#0a0a0f` (near-black, slight cool tint)
- Panel background: `#12121a`
- Primary text: `#e8e8ed`
- Muted text: `#6b7280`
- Accent (primary): `#4a9eff` (calm cyan-blue — the "Jarvis" color)
- Accent (secondary): `#a78bfa` (soft violet — for highlights and Jarvis's voice in transcripts)
- Borders: `rgba(255, 255, 255, 0.06)`

Typography

- **Space Grotesk** — for headers, labels, UI chrome
- **JetBrains Mono** — for transcripts, status indicators, technical readouts
- Load both from Google Fonts
- Use `clamp()` for responsive font scaling

Layout

- Header bar (top): brand wordmark on the left ("JARVIS // Mission Control" in small caps, muted), status indicator on the right ("● IDLE" / "● LISTENING" / etc., in accent color, monospace, tracked-out)
- Main area: two columns — large reactive orb on the left, transcript panel on the right (about 1.5:1 ratio on desktop)
- Footer: text input + send button for typing commands manually

The orb (centerpiece)

A canvas-rendered animated sphere that responds to assistant state:

- **Idle:** slow gentle pulse, ~90px radius oscillating ±3px
- **Listening:** faster pulse, ±15px, slight color shift toward the secondary violet
- **Thinking:** rotating arc/ring around the orb (partial circle that sweeps), inner core slightly contracted
- **Speaking:** larger amplitude pulse (±20px) — ideally tied to actual audio amplitude if feasible; if not, simulated waveform

The orb should have:

- Soft outer glow (radial gradient, accent color, fading to transparent)
- Inner core gradient from secondary accent (top-left) through primary accent to a darker tint at the bottom-right (gives it dimensionality)
- No hard edges — everything should fade smoothly

Use HTML5 Canvas, not SVG. Animate with `requestAnimationFrame`.

Transcript panel

- Monospace font, 13px, generous line-height (~1.7)
- Each message: small uppercase tracked-out role label ("USER" in primary accent, "JARVIS" in secondary accent), then the text below
- Auto-scroll to bottom on new messages
- No avatars, no timestamps in the main view (keep it minimal)

What to avoid

- No emojis in the UI itself
- No glassmorphism / heavy blur (it's overdone)

- No gradient borders or rainbow effects
- No animated backgrounds (particles, mesh gradients, etc.) — the orb is the only animated element
- No icon clutter — text labels where possible

Mobile UI

Same color palette, same typography. Simpler layout — vertical stack of: header, transcript (full width), input + mic button at bottom. No orb on mobile (different format, would feel cramped).

---

7. File / Module Structure

Suggested layout. Adjust if Claude Code finds clear improvements:

```
jarvis/
├── core/              # Core engine (wake word, audio, LLM brain, memory)
├── tools/             # Tool implementations grouped by category
├── ui/                # Static frontend files (HTML/CSS/JS)
├── mobile/            # React Native app (added in Phase 9)
├── data/              # Local data: vector DB, OAuth tokens, credentials
├── server.py          # FastAPI app — main entry point
├── requirements.txt
├── .env.example       # Documents required env vars without leaking values
└── README.md
```

Each tool category file should export:

1. The tool functions themselves
2. A list of (schema_dict, handler_callable) tuples for registration

This keeps tool registration to a single loop in the server.

---

8. Setup Tasks the User Must Do

Claude Code cannot do these — surface them clearly to the user before starting each relevant phase.

Before Phase 1:

- Sign up at console.anthropic.com, create an API key
- Sign up at console.picovoice.ai, create a free access key

- Install FFmpeg (`winget install ffmpeg` / `brew install ffmpeg` / `apt install ffmpeg`)
- On Windows: if pyaudio install fails, use `pip install pipwin && pipwin install pyaudio`
- On macOS: grant terminal Microphone + Accessibility permissions in System Settings
- Add API keys to `.env` (use `.env.example` as template)

Before Phase 5:

- Create a Google Cloud project at console.cloud.google.com
- Enable Gmail API and Calendar API
- Configure OAuth consent screen (External, add self as test user)
- Create OAuth client ID (Desktop app type), download JSON
- Save it to `data/credentials.json`

Before Phase 9:

- Install Expo Go on phone
- Find laptop's local IP via `ipconfig` (Windows) or `ifconfig` (mac/Linux)
- Ensure phone and laptop are on the same Wi-Fi network

---

9. Testing Strategy

Claude Code cannot test voice features (no mic access). Split testing accordingly:

Claude Code can verify:

- Code runs without import errors
- API endpoints return expected JSON
- Tool functions work when called directly with text inputs
- WebSocket handshake succeeds
- UI renders (by checking served HTML)

User must verify manually:

- Wake word detection
- Microphone capture quality

- TTS audio output
- End-to-end voice latency
- Mobile app on physical device

After implementing each phase, Claude Code should:

1. State what was built
2. List specific test commands the user should run
3. List the manual voice tests the user should perform
4. Wait for the user to confirm before moving to the next phase

---

10. Performance Targets

- **End-to-end voice latency:** under 3 seconds from end of user speech to start of TTS response
- **Wake word CPU usage:** under 5% on a modern laptop
- **Memory footprint:** under 1.5 GB total (Whisper "base" model is the biggest contributor)
- **First-token latency from Claude:** target streaming, start TTS on first complete sentence rather than waiting for full response

If a phase introduces regressions on these targets, flag it.

---

11. Security Constraints

- The FastAPI server binds to `0.0.0.0` by default so the mobile app can reach it. Anyone on the local network can hit the endpoints. **Implement token-based auth** in Phase 10 — token in env var, required header on all non-UI routes.
- API keys live only in `.env`, never in code, never in mobile app. The mobile app talks to the laptop, not directly to Anthropic.
- Tools must not allow arbitrary shell execution from voice input — every tool takes specific typed parameters, no "run this command" tool.
- OAuth tokens stored locally in `data/`. Add `data/` to `.gitignore` along with `.env`.

---

12. Cost Awareness

Claude API costs are pay-per-use. Typical usage with Opus runs roughly $0.02–0.08 per command depending on tool-call complexity. For heavy daily personal use, expect $5–15/month. Make the model configurable via env var (`CLAUDE_MODEL`) so the user can switch to a smaller model if needed.

---

13. Working Style for Claude Code

- **Build phase by phase.** Don't skip ahead. Don't try to build all 10 phases in one go.
- **Surface manual setup before writing code** for that phase, so the user can do it in parallel.
- **Default to local/free options** (local Whisper, edge-tts, DuckDuckGo) over paid cloud services unless quality demands otherwise.
- **Code should be readable.** This is a portfolio project — the user will show this code. Prefer clarity over cleverness. Type hints where they help. Docstrings on public functions.
- **Match the user's design taste.** Restrained, professional, no emoji in UI, no over-engineered animations. When in doubt, lean simpler.
- **Don't auto-test voice features.** The user tests those. Verify what you can (imports, type checks, REST endpoints), then hand off.
- **Ask before scope changes.** If you find a reason to deviate from this spec, surface the question before deviating.

---

14. Definition of Done

The project is complete when:

1. User says "Jarvis" → assistant responds within 3 seconds via voice
2. All capability categories work via voice: system control, web tasks, Gmail, Calendar, conversational memory
3. Desktop UI shows real-time state with the reactive orb and live transcripts
4. Mobile app on the user's phone can send commands to the laptop and display responses
5. README documents setup from a fresh clone in under 15 minutes (assuming API keys are ready)
6. The user can demo it to someone else without any "let me just fix one thing" moments

---

15. Out of Scope (for v1)

Do not build these unless the user explicitly asks. They're tempting but will balloon the project:

- Custom wake-word training
- Voice cloning / custom TTS voices
- Smart home integration
- Multi-user support
- Cloud deployment
- Vision / screen-reading capability
- Streaming voice (full-duplex conversation)
- Plugin marketplace
- Web search via paid APIs (Bing, Google)

These are great Phase 2 ideas after v1 is solid.